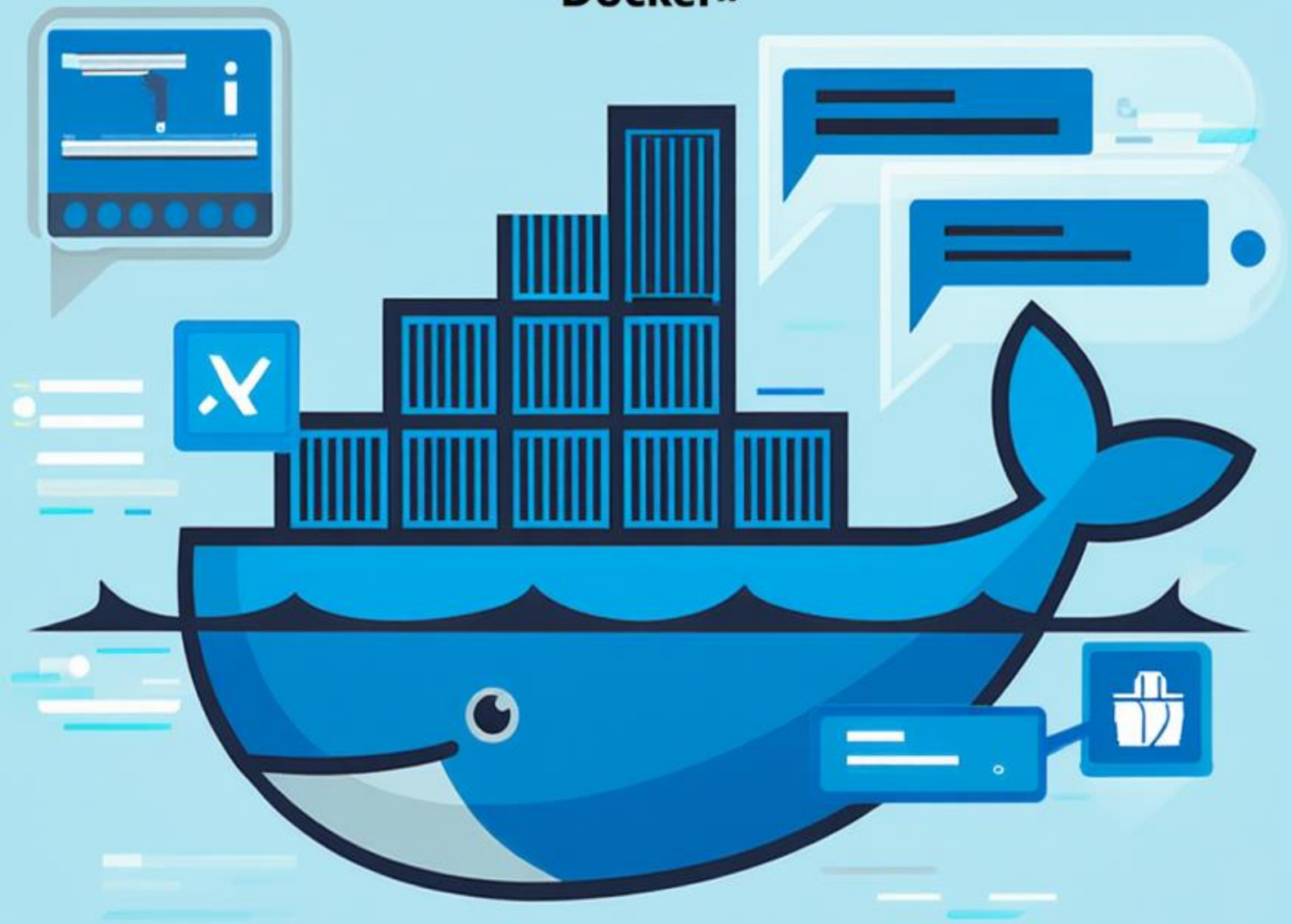
Большая шпаргалка по Docker на базе курса Pragmatic Programmer «Знакомство с Docker»



Pragmatic Programmer

Оглавление

О Шпаргалке ... 3
Основные команды ... 3
Работа с контейнерами ... 3
Работа с образами ... 4
Выполнение команд в контейнере ... 4
Проброс портов ... 4
Dockerfile ... 5
Основные инструкции ... 5
Сборка образа ... 6
Docker Compose ... 7
Базовый docker-compose.yml ... 7
Команды Docker Compose ... 7
Дополнительная Информация ... 8
Информация и логи ... 8
Сети Docker ... 8
Управление сетями ... 8
Типы сетей ... 9
Тома (Volumes) ... 9
Управление томами ... 9
Использование томов ... 10
Полезные команды для разработки ... 10
Просмотр ресурсов ... 10
Очистка системы ... 10
Переменные окружения ... 10
Установка переменных ... 10
Безопасность ... 11
Запуск от имени пользователя ... 11
Ограничение ресурсов ... 11
Практический пример ... 12
Запуск базы данных PostgreSQL ... 12
Всем спасибо ... 16
Полезные ресурсы ... 16

О Шпаргалке

Данная шпаргалка является дополнением и карманным справочником к курсу «Знакомство с Docker». Если будут присутствовать какие-то вопросы, пожелания для развития курса или в целом по новым обучающим материалам, пишите в комментариях к курсам, будем все учитывать. Приятного прочтения и удачи в изучении.

Автор шпаргалки: Pragmatic Programmer

Шпаргалку отредактировал и оформил: Жердев Алексей

Основные команды

Работа с контейнерами

Запуск контейнера

`docker run [ОПЦИИ] IMAGE [КОМАНДА]`

`docker run -d -p 8080:80 --name my-container nginx`

Просмотр контейнеров

`docker ps` # запущенные

`docker ps -a` # все контейнеры

`docker ps -l` # последний контейнер

Управление контейнерами

`docker start CONTAINER` # запустить

`docker stop CONTAINER` # остановить

`docker restart CONTAINER` # перезапустить

`docker pause CONTAINER` # приостановить

`docker unpause CONTAINER` # возобновить

Удаление контейнеров

`docker rm CONTAINER` # удалить остановленный

`docker rm -f CONTAINER` # принудительно удалить

`docker container prune` # удалить все остановленные

Работа с образами

Поиск образов

`docker search TERM`

Загрузка образов

`docker pull IMAGE[:TAG]`

Просмотр образов

`docker images`

`docker image ls`

Удаление образов

`docker rmi IMAGE`

`docker image prune` # удалить неиспользуемые

`docker image prune -a` # удалить все образы

Выполнение команд в контейнере

`docker exec [ОПЦИИ] CONTAINER КОМАНДА`

`docker exec -it my-container bash` # интерактивная оболочка

`docker exec my-container ls -la` # выполнить команду

Проброс портов

Синтаксис проброса портов

`docker run -p HOST_PORT:CONTAINER_PORT nginx`

`docker run -p 8080:80 nginx` # конкретный порт

`docker run -p 80 nginx` # случайный порт хоста

`docker run -p 3000-4000:3000-4000 nginx` # диапазон портов

Dockerfile

Основные инструкции

Базовый образ

```
FROM ubuntu:20.04
```

Метаинформация

```
LABEL maintainer="your@email.com"
```

Установка переменных окружения

```
ENV NODE_ENV=production
```

Копирование файлов

```
COPY . /app
ADD https://example.com/file.tar.gz /tmp/
```

Рабочая директория

```
WORKDIR /app
```

Установка пакетов

```
RUN apt-get update && apt-get install -y nginx
```

Открытие портов

```
EXPOSE 80
```

Точка входа

```
CMD ["nginx", "-g", "daemon off;"]
ENTRYPOINT ["/app/start.sh"]
```

Сборка образа

```
docker build -t my-image:latest .
```

```
docker build -f Dockerfile.dev -t my-app:dev .
```

Docker Compose

Базовый docker-compose.yml

```yaml
version: '3.8'

services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
    volumes:
      - ./html:/usr/share/nginx/html
    depends_on:
      - db

  db:
    image: postgres:13
    environment:
      POSTGRES_PASSWORD: example
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Команды Docker Compose

`docker-compose up` # запуск сервисов

`docker-compose up -d` # запуск в фоне

`docker-compose down` # остановка сервисов

`docker-compose ps` # статус сервисов

`docker-compose logs` # логи всех сервисов

`docker-compose build` # сборка образов

Дополнительная Информация

Далее пошла информация которая не была рассмотрена в курсе, но также является очень полезной.

Информация и логи

Логи контейнеров

`docker logs CONTAINER`

`docker logs -f CONTAINER` # следить за логами

`docker logs --tail N CONTAINER` # последние N строк

Информация о контейнере

`docker inspect CONTAINER`

`docker stats` # статистика в реальном времени

`docker top CONTAINER` # процессы контейнера

Системная информация

`docker version`

`docker info`

Сети Docker

Управление сетями

список сетей

`docker network ls`

создать сеть

`docker network create my-network`

информация о сети

`docker network inspect my-network`

удалить сеть

`docker network rm my-network`

Типы сетей

- `bridge` - стандартная сеть (по умолчанию)
- `host` - использует сеть хоста
- `none` - без сети
- `overlay` - для кластеров Swarm

Тома (Volumes)

Управление томами

список томов

`docker volume ls`

создать том

`docker volume create my-volume`

удалить том

`docker volume rm my-volume`

удалить неиспользуемые тома

`docker volume prune`

Использование томов

Монтирование тома

`docker run -v my-volume:/data nginx`

Монтирование каталога хоста

`docker run -v /host/path:/container/path nginx`

Полезные команды для разработки

Просмотр ресурсов

использование диска

`docker system df`

события в реальном времени

`docker system events`

Очистка системы

`docker system prune` # удалить неиспользуемые ресурсы

`docker system prune -a` # удалить ВСЕ неиспользуемые ресурсы

Переменные окружения

Установка переменных

Через командную строку

`docker run -e VAR=value nginx`

Через файл

`docker run --env-file.env nginx`

Безопасность

Запуск от имени пользователя

```
docker run -u 1000:1000 nginx  # UID:GID
```

Ограничение ресурсов

```
docker run --memory=512m --cpus=1.5 nginx
docker run --memory-swap=1g nginx
```

Практический пример 1

Запуск базы данных PostgreSQL

Команда:

```
docker run -d \
  --name postgres-db \
  -e POSTGRES_PASSWORD=mypassword \
  -v pg_data:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres:13
```

Детальное объяснение каждой опции

`docker run`

- **Назначение**: Основная команда для создания и запуска нового контейнера
- **Альтернативы**: docker create + docker start

`-d (detach)`

- **Назначение**: Запуск контейнера в фоновом режиме
- **Что делает**: Контейнер работает независимо от терминала
- **Без флага**: Контейнер запустится в foreground и заблокирует терминал
- **Проверка**: docker ps покажет работающий контейнер

`--name postgres-db`

- **Назначение**: Присвоение понятного имени контейнеру
- **Что делает**: Создает псевдоним для удобной работы с контейнером
- **Без опции**: Docker сгенерирует случайное имя (например, sleepy_curie)
- **Использование**: Теперь можно обращаться как postgres-db вместо ID

`-e POSTGRES_PASSWORD=mypassword`

- **Назначение**: Установка переменной окружения
- **Что делает**: Задает пароль для пользователя postgres
- **Важность**: Обязательная переменная для образа PostgreSQL
- **Без нее**: Контейнер не запустится (ошибка и выход)
- **Другие переменные**: POSTGRES_USER, POSTGRES_DB

`-v pg_data:/var/lib/postgresql/data`

- **Назначение**: Монтирование тома для сохранения данных
- **Синтаксис**: ИМЯ_ТОМА:ПУТЬ_В_КОНТЕЙНЕРЕ
- **Что делает**:
  - Создает том pg_data (если не существует)
  - Связывает его с каталогом данных PostgreSQL в контейнере
- **Результат**: Данные БД сохраняются между перезапусками контейнера
- **Без тома**: Данные теряются при удалении контейнера

`-p 5432:5432`

- **Назначение**: Проброс портов между хостом и контейнером
- **Синтаксис**: ПОРТ_ХОСТА:ПОРТ_КОНТЕЙНЕРА
- **Что делает**: Открывает доступ к PostgreSQL на localhost:5432
- **Подключение**: psql -h localhost -p 5432 -U postgres
- **Без проброса**: Доступ к БД только изнутри контейнера

`postgres:13`

- **Назначение**: Указание образа для создания контейнера
- **Формат**: ИМЯ_ОБРАЗА:ТЕГ
- **Что делает**:
  - Ищет образ локально
  - Если нет - скачивает из Docker Hub
  - Использует конкретную версию PostgreSQL 13
- **Без тега**: Используется тег latest

Что происходит при выполнении команды

1. **Поиск образа** → Локально или скачивание postgres:13
2. **Создание тома** → pg_data для хранения данных
3. **Запуск контейнера** → В фоновом режиме с именем postgres-db
4. **Настройка БД** → С паролем mypassword
5. **Открытие порта** → Доступ на localhost:5432
6. **Инициализация** → PostgreSQL запускается и готов к работе

Практический пример 2

Запуск базы данных PostgreSQL

Команда:

```
docker run -d \
  --name postgres-db \
  -e POSTGRES_PASSWORD=mypassword \
  -v pg_data:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres:13
```

Детальное объяснение каждой опции

`docker run`

- **Назначение**: Основная команда для создания и запуска нового контейнера
- **Альтернативы**: docker create + docker start

`-d (detach)`

- **Назначение**: Запуск контейнера в фоновом режиме
- **Что делает**: Контейнер работает независимо от терминала
- **Без флага**: Контейнер запустится в foreground и заблокирует терминал
- **Проверка**: docker ps покажет работающий контейнер

`--name postgres-db`

- **Назначение**: Присвоение понятного имени контейнеру
- **Что делает**: Создает псевдоним для удобной работы с контейнером
- **Без опции**: Docker сгенерирует случайное имя (например, sleepy_curie)
- **Использование**: Теперь можно обращаться как postgres-db вместо ID

`-e POSTGRES_PASSWORD=mypassword`

- **Назначение**: Установка переменной окружения
- **Что делает**: Задает пароль для пользователя postgres
- **Важность**: Обязательная переменная для образа PostgreSQL
- **Без нее**: Контейнер не запустится (ошибка и выход)
- **Другие переменные**: POSTGRES_USER, POSTGRES_DB

`-v pg_data:/var/lib/postgresql/data`

- **Назначение**: Монтирование тома для сохранения данных
- **Синтаксис**: ИМЯ_ТОМА:ПУТЬ_В_КОНТЕЙНЕРЕ
- **Что делает**:
  - Создает том pg_data (если не существует)
  - Связывает его с каталогом данных PostgreSQL в контейнере
- **Результат**: Данные БД сохраняются между перезапусками контейнера
- **Без тома**: Данные теряются при удалении контейнера

`-p 5432:5432`

- **Назначение**: Проброс портов между хостом и контейнером
- **Синтаксис**: ПОРТ_ХОСТА:ПОРТ_КОНТЕЙНЕРА
- **Что делает**: Открывает доступ к PostgreSQL на localhost:5432
- **Подключение**: psql -h localhost -p 5432 -U postgres
- **Без проброса**: Доступ к БД только изнутри контейнера

`postgres:13`

- **Назначение**: Указание образа для создания контейнера
- **Формат**: ИМЯ_ОБРАЗА:ТЕГ

- **Что делает**:
    - Ищет образ локально
    - Если нет - скачивает из Docker Hub
    - Использует конкретную версию PostgreSQL 13
- **Без тега**: Используется тег latest

Что происходит при выполнении команды

1. **Поиск образа** → Локально или скачивание postgres:13
2. **Создание тома** → pg_data для хранения данных
3. **Запуск контейнера** → В фоновом режиме с именем postgres-db
4. **Настройка БД** → С паролем mypassword
5. **Открытие порта** → Доступ на localhost:5432
6. **Инициализация** → PostgreSQL запускается и готов к работе

Всем спасибо

Эта шпаргалка покрывает большинство повседневных задач при работе с **Docker**. Хочу всех поблагодарить за окончание курса и не забывать давать обратную связь своими отзывами о данном курсе. Всем удачи в дальнейшем обучении!)

Полезные ресурсы

- Офф. Дока по dockerfile
- Крутая статья по Docker Compose**(habr)**
- Более полный курс по основам Docker